Dual-Clock Asynchronous FIFO

Design, verification and hardware validation of a clock-domain-crossing FIFO in Verilog

Jyotishman Deori

Roll No. 25M1186

M.Tech, Electronic Systems

Indian Institute of Technology Bombay

RESULT SUMMARY

286,176,459
words verified on hardware, zero errors

0
latches inferred; 0 synthesis warnings

100%
functional coverage, 9 assertions clean

1 Overview

A FIFO that moves data between two clock domains with no fixed relationship to each other. The design is Verilog-2001; the verification around it is SystemVerilog. It has been simulated, formally constrained with assertions, synthesised, and finally run on real silicon across two independent crystal oscillators.

The motivation was specific. In my M.Tech project the PS and PL sides of an AXI-Stream interface sit in different clock domains, and I was relying on a handshake somebody else had written. I wanted to build the crossing itself rather than use one.

VERIFICATION SUMMARY

Stage	Tool	Outcome
Functional simulation	ModelSim ASE 20.1	10,017 words, 0 mismatches
Assertions & coverage	Vivado XSim 2020.2	9 properties clean, 100% coverage
Fault injection	both	every seeded bug detected
Synthesis	Vivado 2020.2	0 latches, 0 warnings
Hardware	PYNQ-Z2, Zynq-7020	**286,176,459 words, 0 errors**

2 The problem

Sampling a multi-bit counter in a foreign clock domain can catch it mid-transition. Different bits arrive from different sides of the flop, and the value read is one the counter never actually held. A binary counter stepping `0111 → 1000` flips four bits at once; sampled at the wrong instant it can return anything between `0000` and `1111`.

Two mechanisms fix this, and both are needed:

- **Gray coding** makes consecutive values differ in exactly one bit, so a bad sample returns either the old value or the new one, never a third.
- **Two-flop synchronisers** give a metastable first flop a full clock period to settle before anything downstream looks at it.

They solve different problems. Gray coding does nothing about metastability; synchronisers do nothing about multi-bit skew. Omit either and the design is broken in a way simulation may never show you — a claim this report later tests directly, and which turned out to be more literally true than expected.

3 Architecture

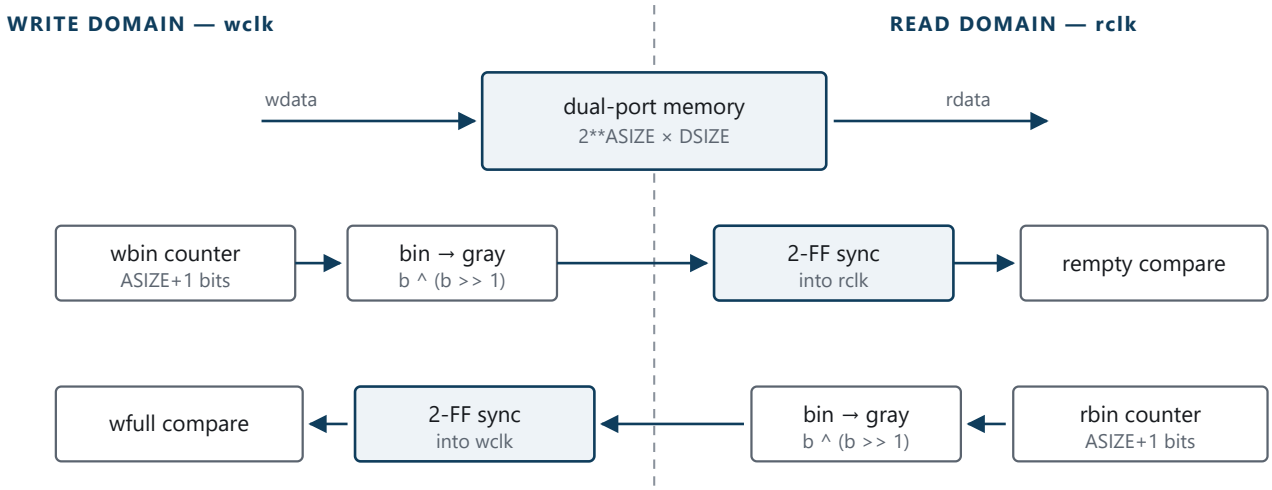

Figure 1 — Each pointer crosses once, in one direction only. The binary counters stay in their own domain; only their Gray-coded copies cross, and each crossing passes through a two-flop synchroniser.

The design is two files. `rtl/sync_2ff.v` is the synchroniser, kept as a separate module so that it is visibly present rather than buried; `rtl/async_fifo.v` is everything else. The structure follows Cliff Cummings, *Simulation and Synthesis Techniques for Asynchronous FIFO Design* (SNUG 2002).

```
module async_fifo #(
    parameter DSIZE = 8,
    parameter ASIZE = 4               // depth = 2**ASIZE
) (
    input  wire             wclk, wrst_n, winc,
    input  wire [DSIZE-1:0] wdata,
    output reg              wfull,

    input  wire             rclk, rrst_n, rinc,
    output wire [DSIZE-1:0] rdata,
    output reg              rempty
);
```

3.1 The parts that are easy to get wrong

Pointers are one bit wider than the address. With `ASIZE` address bits the pointers are `ASIZE+1` bits. That extra bit is the only thing distinguishing full from empty, since in both cases the addresses are equal.

Empty is when the read pointer has caught up, so every bit matches:

```
rempty = (rgray_next == rq2_wptr);
```

Full is when the write pointer has wrapped once more than the read pointer and caught it. Same low bits, but the top *two* bits inverted:

```
wfull = (wgray_next ==
         {~wq2_rptr[ASIZE], ~wq2_rptr[ASIZE-1], wq2_rptr[ASIZE-2:0]});
```

Two bits, not one. In Gray code both the MSB and the bit beneath it flip when the counter passes the halfway point, so comparing with only the top bit inverted detects the halfway crossing rather than the wrap. This is the single most likely place to introduce a bug, and it is the fault deliberately injected in §5.3.

Both flags are computed from the *next* pointer value rather than the registered one, so they assert on the same edge the FIFO actually fills or empties instead of one cycle late.

> **LOOKS LIKE A BUG, IS NOT**
>
> Each side only ever sees a copy of the other pointer that is two clocks stale, so `wfull` can remain asserted briefly after a read has freed space, and `rempty` briefly after a write. The FIFO reports itself fuller or emptier than it truly is. It never errs in the opposite direction, which is the only one that could cause overflow or underflow.

4 Verification strategy

One directed test passing says almost nothing about a CDC design. What finds bugs is stalling both interfaces at random and running the clocks at a ratio the design was not written for.

4.1 Simulation

The testbench runs two phases in a single simulation: fast write against slow read (7 ns / 11 ns), then the reverse. Periods of 7 and 11 are deliberate; related periods such as 10 and 20 align their edges and conceal exactly the class of bug being hunted. A reference queue is pushed on the write side and popped and compared on the read side, with both enables randomised so that full and empty are genuinely reached.

Each phase ends by stopping the writes and reading until the FIFO is empty, so that every word written is compared on the way out. Without that drain, several hundred words remain in the FIFO at the end of a phase and are never checked.

MODELSIM ASE 20.1 — FUNCTIONAL TESTBENCH

Clock ratios exercised	7/11 ns and 11/7 ns
Words written and compared	10,017
Data mismatches	**0**
Overflows / underflows	**0 / 0**
Words left unchecked	**0**
Cycles spent full / empty	12,678 / 12,419
Back-to-back writes	3,025
Concurrent read and write	3,546

4.2 Assertions and functional coverage

ModelSim ASE supports neither SVA nor covergroups, so the assertion layer runs under Vivado XSim. The checker is `bind`-ed onto the DUT rather than written inside it, keeping the synthesisable RTL free of verification code.

Nine properties are checked. The most important is the Gray property itself, which directly tests the premise the whole design rests on:

```
a_wgray_one_bit: assert property (
    @(posedge wclk) disable iff (!wrst_n)
    $countones(wgray ^ $past(wgray)) <= 1
);
```

Others confirm that occupancy never exceeds the depth (which, in modular pointer arithmetic, catches underflow as well as overflow), that the flags genuinely gate the pointers, that pointers only ever hold or step by one, and that the reset state is correct.

VIVADO XSIM 2020.2 — ASSERTIONS AND COVERAGE

Words written and compared	10,015
Assertion failures across 9 properties	**0**
Write interface coverage	100%
Read interface coverage	100%
Occupancy coverage (empty and full bins)	100%
Concurrent access coverage	100%

The word counts differ by two between simulators because the stimulus is randomised and the two implementations of `$urandom` do not produce the same stream from the same seed.

4.3 Fault injection

A test that has never failed is not evidence. Before any of the numbers above were trusted, the design was broken deliberately and everything re-run.

SEEDED FAULTS AND WHAT CAUGHT THEM

Injected fault	Detected by
Full comparison with only the MSB inverted	Occupancy check and scoreboard, within 500 ns
The same fault, under XSim	`a_no_overflow` : *occupancy 17 exceeds depth 16*
Gray coding removed entirely	**Nothing. The full testbench passed.**

That last result is the most interesting outcome of the whole project and is the subject of the next section.

5 Does the Gray coding actually do anything?

With the crossing switched to raw binary pointers — using the matching binary full and empty tests, so that the encoding was the only variable — the entire testbench passed. Zero mismatches, both clock ratios, full and empty both reached.

The reason is that in RTL simulation every bit of a bus changes at exactly the same instant. There is no skew, so there is no window in which a sampler can catch part of an old value and part of a new one. The hazard Gray coding exists to prevent is invisible at this level of abstraction.

Demonstrating it therefore required injecting per-bit transport delay on the crossing bus by hand, and counting how often a domain latched that bus while it was still settling. Two corrections were needed before the experiment measured anything at all:

- **The delay pattern must not be monotonic.** Delaying bit i by $i \times$ step means the low bits always move first, so the transient value is always *smaller* than both the old and new pointer. That direction is harmless: a pointer reading low merely makes the other side believe the FIFO is fuller, or emptier, than it is, which is the pessimistic direction the design already tolerates.

- **The clocks must not be rationally related.** With 7 ns and 11 ns, a read edge only ever falls a whole number of nanoseconds after a write edge — eleven fixed sampling phases, which step over most of the sub-nanosecond windows of interest. Changing the read clock to 11.3 ns lets the phase drift through everything, as two real oscillators do.

SKEW SWEEP — 4,000 TRANSACTIONS PER CASE, WCLK 7.0 NS, RCLK 11.3 NS

Skew (ns)	Gray caught mid-flight	Gray errors	Binary caught mid-flight	Binary errors
0.0	0	**0**	0	0
0.4	413	**0**	649	0
1.0	895	**0**	1,391	0
2.0	1,840	**0**	2,505	0
3.5	3,234	**0**	5,785	**3,513**

Skew is quoted as the per-bit step; worst-case spread across the bus is three times that figure. Gray remains at zero errors throughout. In the final row it was caught mid-transition 3,234 times and still did not corrupt or drop a single word, because only one bit ever moves: catch it early or late and the result is the old value or the new one, and both are safe.

5.1 Why binary survives small skew

Binary needs more explanation, since it survives the first three rows despite thousands of mid-transition samples. A transient only exists while a pointer is *moving*, and a pointer moves only when that side has actually done something. If the write domain latches a corrupted read pointer, the corruption is itself proof that a read just occurred, which means a slot genuinely was freed. The garbage breaks the equality test for exactly one cycle, so at most one extra write slips through, and that write had somewhere to go.

The argument holds only while the transient is shorter than a destination clock period, so that just one sampling edge can fall inside it. At 3.5 ns the bus spread is 10.5 ns against a 7 ns write clock; the bad value now survives across consecutive edges, several operations slip through per event, and the self-limiting property collapses. 89% of words read came back wrong.

> **CONCLUSION**
>
> Binary pointers are not merely riskier than Gray. They are correct only while skew stays below a bound nobody checks, on a path that is by definition unconstrained, and they fail completely rather than gracefully once that bound is crossed. Gray coding has no such bound: the guarantee is structural rather than a matter of degree.

6 Synthesis

Synthesised out of context for `xc7z020clg400-1`. The purpose is not a bitstream but three questions: were any latches inferred, is the size sane, and did the tool recognise both crossings.

VIVADO 2020.2 — OUT-OF-CONTEXT SYNTHESIS

Errors / warnings / critical warnings	**0 / 0 / 0**
Latches inferred	**0**
Slice LUTs	28 (20 logic, 8 distributed RAM)
Slice registers	40, all flip-flops
Block RAM / DSP	0 / 0

The 16×8 memory was mapped to distributed RAM rather than a block RAM, which is expected at this size; a BRAM would sit almost entirely empty.

Vivado's CDC report locates both crossings, at depth 2, carrying the `ASYNC_REG` property and covered by the asynchronous clock group:

```
CDC-3  Info      2   1-bit synchronized with ASYNC_REG property
CDC-6  Warning   2   Multi-bit synchronized with ASYNC_REG property
```

The CDC-6 warnings are correct, and their absence would be more concerning than their presence. Vivado can see a multi-bit bus entering a two-flop synchroniser, which is normally a genuine bug, and nothing tells it that this particular bus is Gray coded.

> **UNPLANNED OBSERVATION**
>
> The report sources the top pointer bit from `wbin_reg[4]` rather than `wgray_reg[4]`. The MSB of a Gray code equals the MSB of the binary it was derived from, so `bin ^ (bin >> 1)` leaves that bit untouched and synthesis simply removed the duplicate register.

7 Hardware validation on a PYNQ-Z2

Simulation said the design worked. Simulation also said it worked with the Gray coding removed, so the final step was real silicon.

7.1 Why the clock choice matters

The value of a hardware run lies entirely in the clocks, so that is the part worth getting right. The straightforward approach — take the board's 125 MHz oscillator and derive a second frequency from it with an MMCM — would have been substantially less work and would have proved very little. Two clocks from one MMCM are phase

locked: the relationship between their edges is fixed and repeats, so sampling occurs at the same handful of relative phases indefinitely and the synchroniser may never once be caught inside its setup/hold window. It would resemble a hardware test while demonstrating less than the simulation already had.

Instead `wclk` is `FCLK_CLK0`, derived from the PS PLL and the 33.333 MHz crystal, while `rclk` is the separate 125 MHz board oscillator. Two crystals, no common reference. They drift against one another continuously, so across a ten-second run the sampling point sweeps through every phase relationship there is, including those that land a synchroniser input precisely on its edge. The Zynq PS is present in the design for this reason alone; nothing else in it is used.

7.2 The test harness

The simulation testbench rebuilt as synthesisable logic: an LFSR generates the write data, an identical LFSR on the read side predicts what should return, and a comparator counts words that do not match. A second pair of LFSRs stalls both interfaces so that full and empty are reached. Counters are read back over JTAG through a VIO core.

If the FIFO ever drops or duplicates a single word, the read side's replica falls permanently out of step and every subsequent word counts as an error. A one-word failure therefore appears as tens of millions of errors rather than as one, which is the desired behaviour from a checker.

PYNQ-Z2 (XC7Z020CLG400-1) — TWO PHASES, TEN SECONDS EACH

Phase	Written	Read	Errors
1 — write fast, read slow	159,905,237	159,905,221	**0**
2 — write slow, read fast	286,176,459	286,176,459	**0**

HARDWARE RESULT

Words written and checked	286,176,459
Data errors	**0**
Reached full / empty	**yes / yes**
Worst setup slack (WNS)	1.997 ns
Worst hold slack (WHS)	0.061 ns

> **THE RESIDUAL IS THE DEPTH**
>
> Phase 1 finishes sixteen words short of what it wrote, and the FIFO is exactly sixteen deep. The phase stops with the writer running flat out against a slow reader, so the FIFO is full at the instant writes cease and is still holding a full sixteen words that have not been read. Phase 2 inverts the rates, the reader drains them and then keeps pace, and the totals finish exactly equal. The difference is not noise; it is the depth.

For scale, the simulation checked 10,017 words. The hardware run checked roughly 28,000 times more, on real routing with real metastability, in twenty seconds.

7.3 A fault found in the harness

> **CDC-10 — COMBINATIONAL LOGIC BEFORE A SYNCHRONISER**
>
> The first build reported a path from the VIO's output register directly to `wrst_sync_reg[0]/CLR`. The reset had been written as a single line, `arst_n = fclk_rst_n && !vio_rst`, feeding both reset synchronisers. That AND gate sat on the asynchronous clear pin of a synchroniser flop, where a glitch would reset one clock domain and not the other, leaving the FIFO's two pointers disagreeing about where the data begins. That is precisely the failure this project exists to prevent, occurring inside the harness built to demonstrate it does not happen.
>
> The reset is now built in three stages: `fclk_rst_n` is the only asynchronous reset and has no logic in front of it, the soft reset crosses into the write domain through a `sync_2ff` like any other control bit, and the two are combined and registered before anything downstream sees them.

The full-design CDC report also raises 32 CDC-1 criticals, all running from the FIFO memory to the comparator. These are correct as they stand: data is written on `wclk` and read on `rclk` with no synchroniser in between, which is exactly what a CDC checker should flag, except that the pointer protocol guarantees the read side only ever addresses words written at least two synchroniser stages earlier. The data has been stationary for several clocks by the time anything reads it. Vivado can see wires, not protocols. Notably this did not appear in the out-of-context run, because there `rdata` terminates at a port with no consumer logic to trace into.

8 What this does not prove

The skew experiment models bus skew, not metastability. Its synchroniser is two ideal flops that resolve instantly, so nothing in it speaks to MTBF, and no amount of Gray coding removes the need for the synchroniser. The two mechanisms address different problems and only one of them is exercised there.

The hardware run says nothing about MTBF either. Twenty seconds of two drifting clocks is a thorough sweep of the phase space but it is not a synchroniser reliability measurement, and a two-flop synchroniser has a finite failure rate that a run of this length would not expose. Nothing here justifies two flops over three at a higher frequency; that is an MTBF calculation, not a test.

9 Reproducing the results

```
# functional testbench (ModelSim)
export PATH="/c/intelFPGA_lite/20.1/modelsim_ase/win32aloem:$PATH"
cd sim && vsim -c -do run.do

# assertions and functional coverage (Vivado XSim)
cd sim && ./run_xsim.sh

# latch and CDC check
export PATH="/c/Xilinx/Vivado/2020.2/bin:$PATH"
vivado -mode batch -source synth/synth_check.tcl

# the CDC encoding experiment
cd sim && vsim -c -do run_experiment.do

# hardware: build, program, run
vivado -mode batch -source hw/build.tcl
xsdb hw/program.tcl
vivado -mode batch -source hw/run_hw_test.tcl
```

Raw transcripts for every figure quoted in this report are committed under `results/`. No number appears here that was not read out of one of them.

REPOSITORY LAYOUT

`rtl/`	the design; the only thing synthesised
`tb/`	testbench, plus the bound SVA and covergroup checker
`sim/`	ModelSim and XSim run scripts
`synth/`	Vivado latch and CDC check
`hw/`	PYNQ-Z2 build, programming and hardware test
`experiments/`	deliberately broken variants, kept for what they showed
`results/`	raw transcripts of every run quoted